



```
load_dataset('iris') directly from seaborn
```

1. Basic Data Exploration

DSA103 Advanced Chemical Data Science

Lecture 7: Exploratory Data Analysis - finding patterns in data

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

```
int(iris.head())
```

Summary of dataset

```
int("\nDataset Info:")
```

```
int(iris.info())
```

Statistical summary

```
Statistical Summary:")
```

	sepal length (cm)	petal length (cm)	species
5	4.7	3.2	1.3, 0.2, 0, setosa
6	4.6	3.1	1.5, 0.2, 0, setosa
7	5.0	3.6	1.4, 0.2, 0, setosa
8	5.4	3.9	1.7, 0.4, 0, setosa
9	4.6	3.4	1.4, 0.3, 0, setosa
10	5.0	3.4	1.5, 0.2, 0, setosa
11	4.4	2.9	1.4, 0.2, 0, setosa
12	4.9	3.1	1.5, 0.1, 0, setosa
13	5.4	3.7	1.5, 0.2, 0, setosa
14	4.8	3.4	1.6, 0.2, 0, setosa
15	4.8	3.0	1.4, 0.1, 0, setosa
16	4.3	3.0	1.1, 0.1, 0, setosa
17	5.8	4.0	1.2, 0.2, 0, setosa
18	5.7	4.4	1.5, 0.4, 0, setosa
19	5.4	3.9	1.3, 0.4, 0, setosa
20	5.1	3.5	1.4, 0.3, 0, setosa
21	5.7	3.8	1.7, 0.3, 0, setosa
22	5.1	3.8	1.5, 0.3, 0, setosa
23	5.4	3.4	1.7, 0.2, 0, setosa
24	5.1	3.7	1.5, 0.4, 0, setosa
25	4.6	3.6	1.0, 0.2, 0, setosa
26	5.1	3.3	1.7, 0.5, 0, setosa
27	4.8	3.4	1.9, 0.2, 0, setosa
28	5.0	3.0	1.6, 0.2, 0, setosa
29	5.0	3.4	1.6, 0.4, 0, setosa

Content overview

Recap Pandas

What is EDA, goals of EDA?

Tools for EDA:

- Inspect data with pandas: head, shape, info(), nunique, nlargest, sort, value_counts, ...
- Visualisation as vital tool in EDA (matplotlib, seaborn)
- Dimensionality Reduction, PCA

Activity: EDA on iris data set

Pandas

Built on NumPy, i.e. also Pandas objects are typed (both keys and values)

More flexible to work with data: attaching labels, working with missing data...

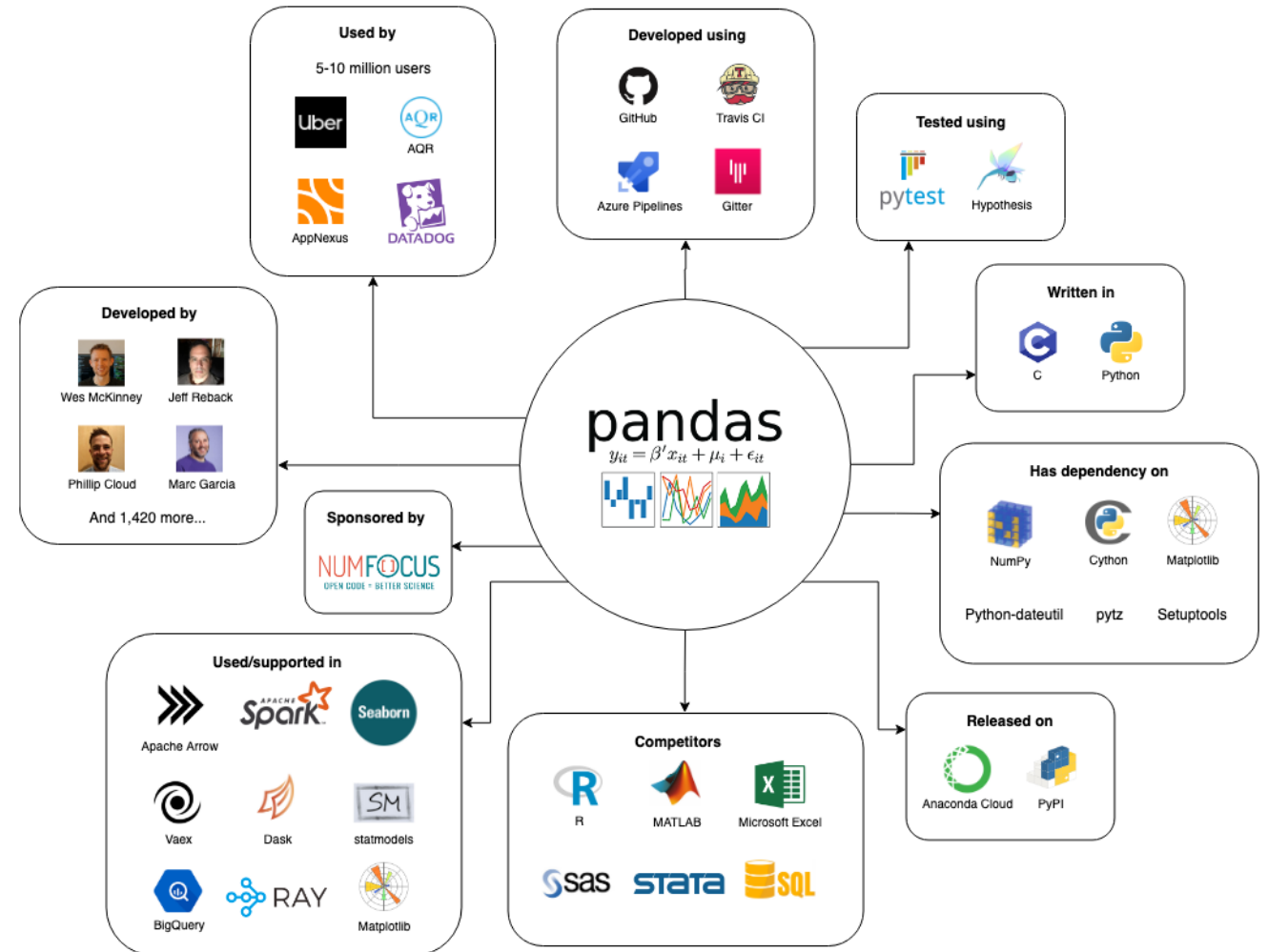
Pandas objects are basically “enhanced” numpy arrays:
Series (1D), **DataFrames** (2D)

```
import pandas as pd
pd.__version__
```

✓ 1.7s

'2.3.3'

<https://www.w3schools.com/python/pandas/default.asp>
<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>
https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html



https://se.ewi.tudelft.nl/desosa2019/chapters/pandas/images/pandas/context_view.png

Pandas – Series

A Pandas **Series** object is a one-dimensional array of indexed data

- NumPy array: has an *implicitly* defined integer index
- A **Series** object uses by default integer indices
- A **Series** object can have an *explicitly* defined index associated with the values:
- The index labels can be accessed by the **index** attribute:
- If a **Series** is created from a dictionary, the index will be the keys of the dict:

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

```
data1 = pd.Series([1, 2, 3, 4, 5])  
✓ 0.0s
```

```
data2 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])  
✓ 0.0s
```

```
d2index = data2.index  
print(d2index)  
✓ 0.0s  
Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

```
data3 = pd.Series({'b': 1, 'e': 2, 'c': 3, 'd': 4, 'a': 5})  
print(data3.index)  
✓ 0.0s  
Index(['b', 'e', 'c', 'd', 'a'], dtype='object')
```

Pandas - DataFrame

- A **DataFrame** object is an analog of a two-dimensional array both with flexible row indices and flexible column names
- Both the rows and columns have a generalized index for accessing the data
- The row indices can be accessed by using the **index** attribute
- The column indices can be accessed by using the **columns** attribute
- **DataFrame** = sequence of aligned **Series** objects (column = **Series**)

columns (axis 1)

	foo	bar
a	0.652252	0.324959
b	0.731989	0.088452
c	0.557951	0.729868

index (axis 0)

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas – Constructing a DataFrame

- From a single Series object:

```
population = pd.Series([1.4, 1.3, 0.3, 0.2], index=['China', 'India', 'USA', 'Indonesia'])
pd.DataFrame(population, columns=['Population (in billions)'])
```

✓ 0.0s

- From a list of dictionaries:

```
pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

✓ 0.0s

- From a dictionary of Series objects:

```
area = pd.Series({'California': 423967, 'Texas': 695662,
                  'New York': 141297, 'Florida': 170312,
                  'Illinois': 149995})
```

```
population = pd.Series({'California': 38332521, 'Texas': 26448193,
                        'New York': 19651127, 'Florida': 19552860,
                        'Illinois': 12882135})
```

```
data = pd.DataFrame({'area': area, 'population': population})
```

- From a two-dimensional NumPy array:

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Import, e.g. from URL, CSV, etc.

```
pd.DataFrame(np.random.rand(3, 2),
             columns=['foo', 'bar'],
             index=['a', 'b', 'c'])
```

✓ 0.0s

	foo	bar
a	0.652252	0.324959
b	0.731989	0.088452
c	0.557951	0.729868

Pandas – Data Selection: DataFrame

DataFrame can be treated as a **dictionary of related Series** objects:

- Select Series by the column name, e.g. **df['area']**
- Modify the **DataFrame** object with familiar syntax, e.g. **df['c3'] = df['c2']/ df['c1']**

or:

DataFrame can be treated as **two-dimensional array**:

- Access the underlying NumPy data array by using the **values** attribute, e.g. **df.values[0]** will select the first row
- Use the **iloc** indexer to index, slice, and modify the data by using the *implicit* integer index
- Use the **loc** indexer to index, slice, and modify the data by using the *explicit* index

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas – Adding and removing columns to a DataFrame

- Some methods to add columns:
 - Columns can be added by directly declaring a list as column: **df["new_column"] = values**
 - This is similar to the **assign()** method: **df.assign(name = [values])**
 - To add new columns at arbitrary positions, not just at the end, use the **insert()** method: **df.insert(loc, column, value, allow_duplicates)**
 - Adding multiple columns can be done by merging with another DataFrame using **pd.concat([df1, df2], axis=1)**
- Some methods to remove columns:
 - By name: **df.drop(["col names"], axis = 1, inplace=True)**, where **inplace** specifies if a copy is created or the df is changed directly
 - By index: **df.drop(df.columns[[0, 1, 3]], axis=1, inplace=True)**
 - (Slicing)

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas – Adding and removing rows to a DataFrame

- Some methods to add rows:
 - Directly defining a new row with the **.loc** indexer: **df.loc[len(df)] = [values]** (avoid when working with explicit indices!)
 - Using **_append()** to add a new row from a dictionary (**matching keys!**): **df = df._append({'col1': 'a', 'col2': 1234, 'col3': 23.45}, ignore_index=True)**
 - The **_append()** method can also be used to add a **Series**
 - Use **concat([df1, df2])** to merge with another DataFrame to add multiple lines
- Rows can be removed using the **drop()** method:
 - By label (explicit index): **df.drop("Entry1", inplace=True)**
 - By index: **df.drop(0, inplace=True)** or **df.drop(df.index[0], inplace = True)**

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas ufuncs

Pandas is designed to work with Numpy, hence any NumPy ufunc will work on Pandas **Series** and **DataFrame** objects

- *Index preservation*: Indices are preserved when a new Pandas object will come out after applying ufuncs
- Missing data is marked with **NaN** ("Not a Number", can be checked using **isnull()** or **isna()** function)
- We can specify on how to fill value for any elements that might be missing by using the option **fill_value**: **A.add(B, fill_value=0)**
- We can also use the **dropna()** method to drop entries with missing values
- *Note*: Any of the ufuncs discussed for NumPy can be used in a similar manner with Pandas objects

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas - Aggregation

- As with one-dimensional NumPy array, for a Pandas **Series** aggregates return a single value
- For a **DataFrame**, the aggregates return by default results within each column
- Pandas **Series** and **DataFrames** include all common NumPy aggregates
- In addition, there is a convenient method **describe()** yielding several common aggregates for each column and returns the result
- The **aggregate()** method can be applied for multiple operations, specifying operation and column explicitly (e.g. {"A": "sum", "B": "mean"})
- The **filter()** method allows you to drop data based on group properties *Note: **filter()** takes as an argument a *function* that returns a Boolean value specifying whether the group passes the filtering*
- While aggregation must return a reduced version of the data, **transform()** can return some transformed version of the full data to recombine (meaning that we still have the same number of entries before and after the transformation)
- The **apply()** method applies an arbitrary function to the group results. The function should take a **DataFrame**, and return either a Pandas object or a scalar

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Pandas – Reading and Writing

Read from file:

- Pandas provides the **pd.read_csv()** function to load data from a CSV file from a specified path (locally or URL)
- An optional argument **error_bad_lines** can be set to **False** so that bad lines in the file are ignored when read
- More optional arguments in the documentation: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.read_csv.html

Write to file:

- The **to_csv()** method exports a **DataFrame** to a CSV file
- More details and options: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.to_csv.html

Of course, CSV files are not the only file that Pandas can process. More methods to import and export data in the documentation: <https://pandas.pydata.org/pandas-docs/stable/api.html#id12>

<https://www.w3schools.com/python/pandas/default.asp>

<https://www.geeksforgeeks.org/pandas/pandas-tutorial/>

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

Exercises: Pandas

In pairs of two work through the Pandas puzzles:

- Arrangement for the exercises 5-17: Person A works on the uneven numbers, Person B on the even numbers
- Define the coworking rules (branches, file names, ...) for your repository prior to starting!

Recap pandas – pandas puzzles

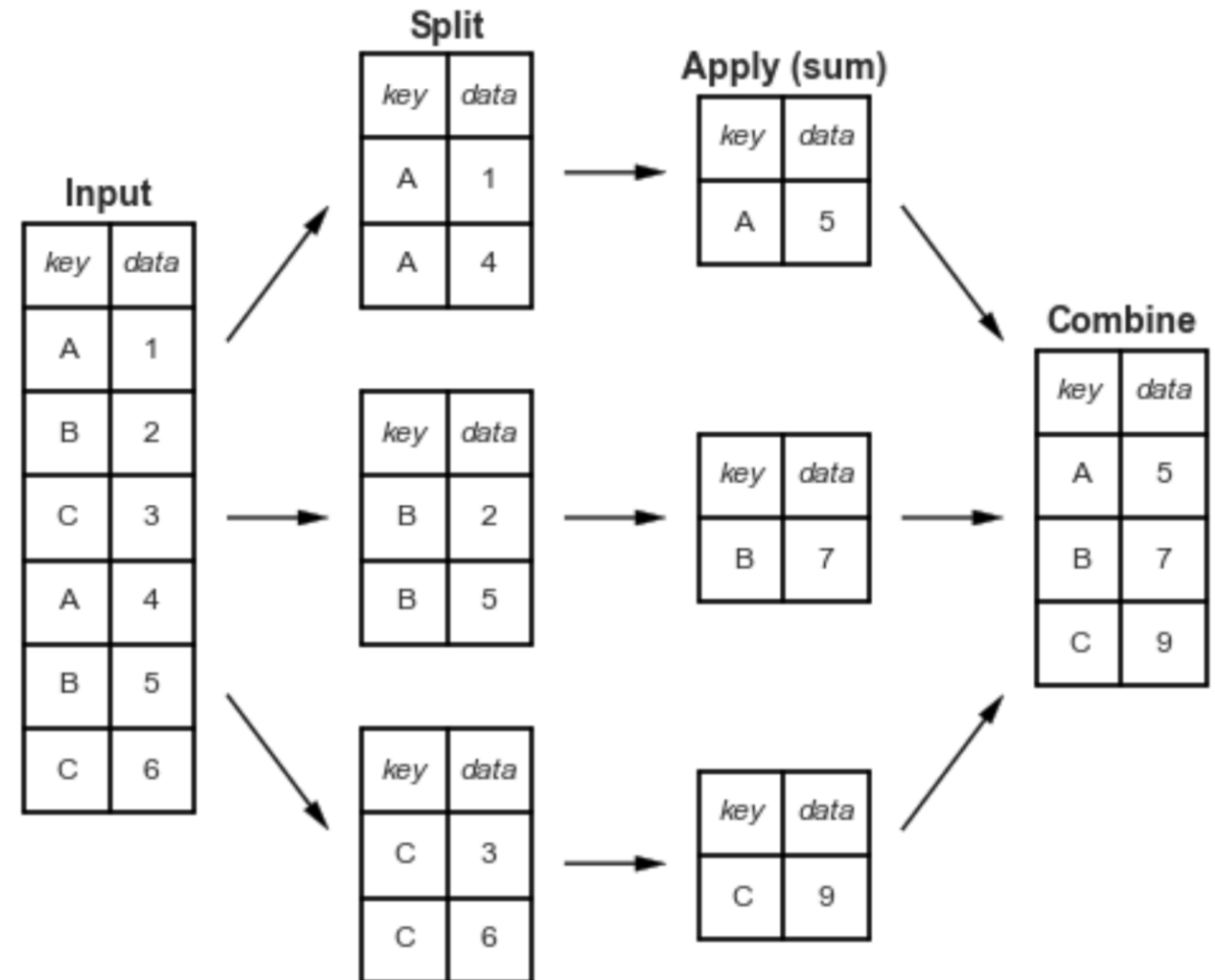
- Selecting data
- Adding and removing data
- Changing column names
- Aggregates
- Groupby

Recap pandas – Split, Apply, Combine

- *Split*: Break up and group a **DataFrame** depending on the value of the specified key
- *Apply*: Apply some function, usually an aggregate, transformation, or filtering, within the individual groups
- *Combine*: Merge the results of these operations into an output array

The **groupBy()** method of **DataFrames** can compute most split-apply-combine operations

Note: `groupBy()` does not return a `DataFrame` but a `DataFrameGroupBy` object (i.e. a “special” view of the `DataFrame`). To this, aggregates can be applied, which will perform the appropriate apply/combine steps to produce the desired result.



<https://jakevdp.github.io/PythonDataScienceHandbook/03.08-aggregation-and-grouping.html>

What is EDA

Exploratory data analysis (EDA) helps to understand what information is contained in a dataset and what to do with the data. It is all about the **characteristics** in the dataset, detecting **anomalies**, finding (hidden) **patterns** and **relationships**.

Goals:

- Suggest hypotheses about topics of interest
- Check if all data is available needed to test the hypotheses
- Help selecting appropriate methods and models for working with the data
- Suggest what data should be gathered for further investigation

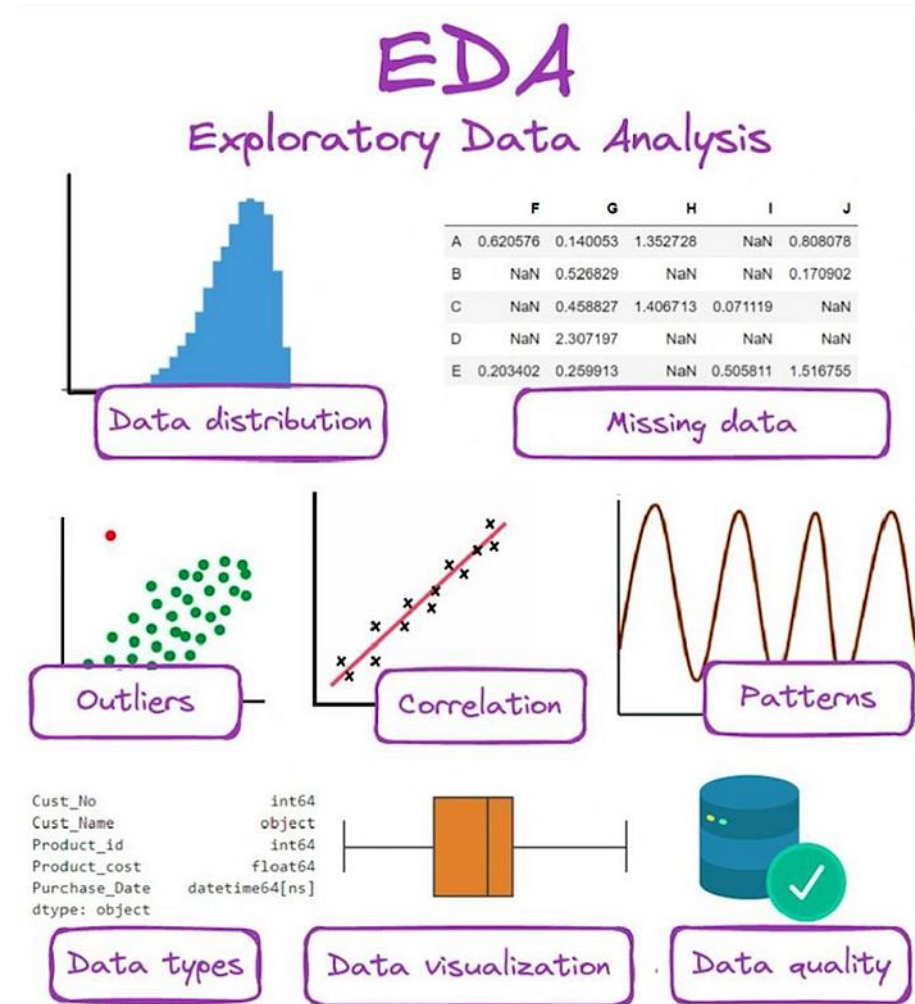
https://lewtun.github.io/dslectures/lesson02_exploratory-data-analysis/#What-is-exploratory-data-analysis?

<https://towardsdatascience.com/a-data-scientists-essential-guide-to-exploratory-data-analysis-25637eee0cf6/>

EDA – step by step

”Look – Summarize – Visualize – Hypothesize

- 1) Understanding the data
- 2) Importing and inspecting the data
- 3) Handle missing data
- 4) Exploring data characteristics
- 5) Performing data transformations
- 6) Visualizing relationships in data
- 7) Assessing outliers
- 8) Communicate findings



Picture taken from:

https://miro.medium.com/v2/resize:fit:1358/1*b2XF4Fx_p_bkz8YA4DJpNg.png

Types and tools of EDA

- **Univariate Analysis:** Find patterns within a single feature, some tools are:
 - Summary statistics (mean, median, variance, standard deviation)
 - Box plots
 - Histogram plots
- **Bivariate Analysis:** Relationships between two variables of interest. Tools include:
 - Scatter plots
 - Correlation assessment
 - Line graphs
 - Covariance analysis
- **Multivariate Analysis:** identify relationships between two or more variables. Important tools include:
 - Pair plots
 - Dimensionality reduction, e.g. PCA

Activity: EDA on iris dataset

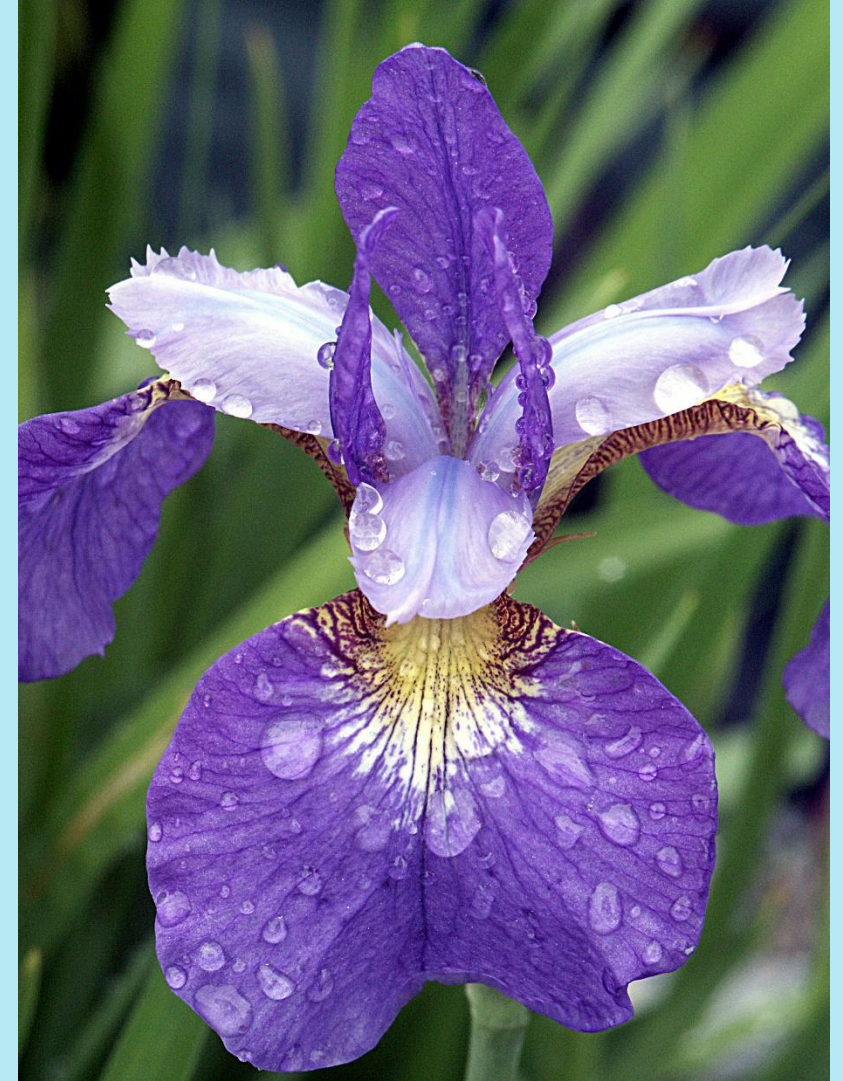
Classic data set for classification models

Data recorded in 1935 by E. Anderson, assembled in 1936 by R. Fisher

50 samples from 3 different flowers with 4 different features recorded each

Tasks:

- 1) Import and load data (DSA_iris_dataset)
- 2) Get an **overview**: head, size, shape, info
- 3) Do some **quick statistics** and **summary**: value_counts, describe, nlargest, nsmallest, nunique
- 4) Look if data is **missing** (isna, notnull) or if there are **duplicates** (duplicated)
- 5) Drop unnecessary columns, rename columns if necessary and treat anomalies
- 6) Export the cleaned dataset as “DSA_iris_cleaned.csv” (same folder!)



E. Anderson, “The Species Problem in Iris.”, *Annals of the Missouri Botanical Garden* **1936**, 23, 457–509, <https://doi.org/10.2307/2394164>.

R. A. Fisher, “The Use of Multiple Measurements in Taxonomic Problems”, *Annals of Eugenics* **1936**, 7, 179-188, <https://doi.org/10.1111/j.1469-1809.1936.tb02137.x>

Visualisation – brief excursion

Visualisation is crucial in understanding data and finding patterns as well as anomalies!

We will be using the libraries **matplotlib** (alias: **plt**) and **seaborn** (alias: **sns**)

We will use the following plots in this sessions:

- **Histograms**
- **Box plots**
- **Scatter plots**
- **Pair plots**
- **Heatmap**

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Visualisation – Histogram

A histogram shows the statistical distribution of the features.

– Matplotlib:

E.g. plot for all features

```
# matplotlib histogram
df.hist(bins=5) # bins = number of bars
plt.suptitle('Histograms of All Features', fontsize=16)
plt.show()
```

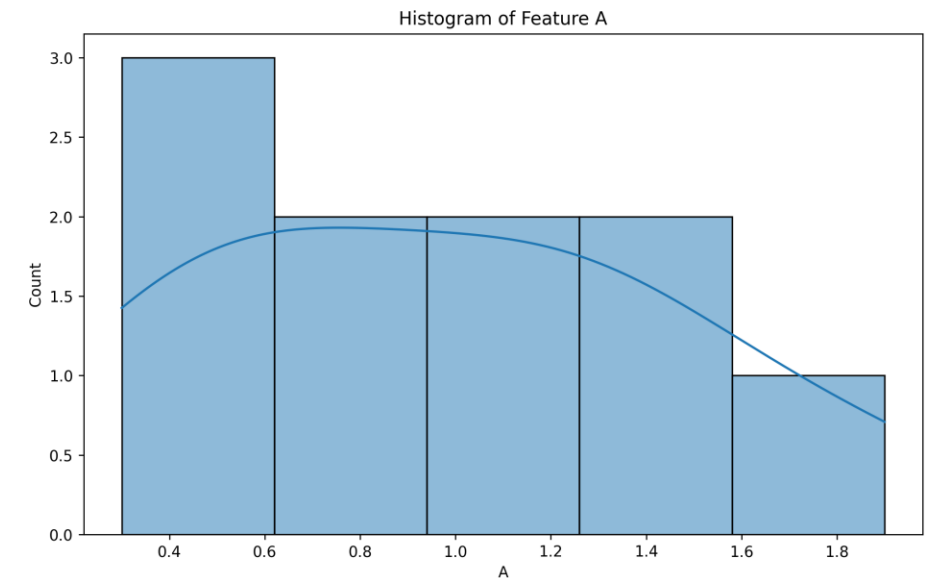
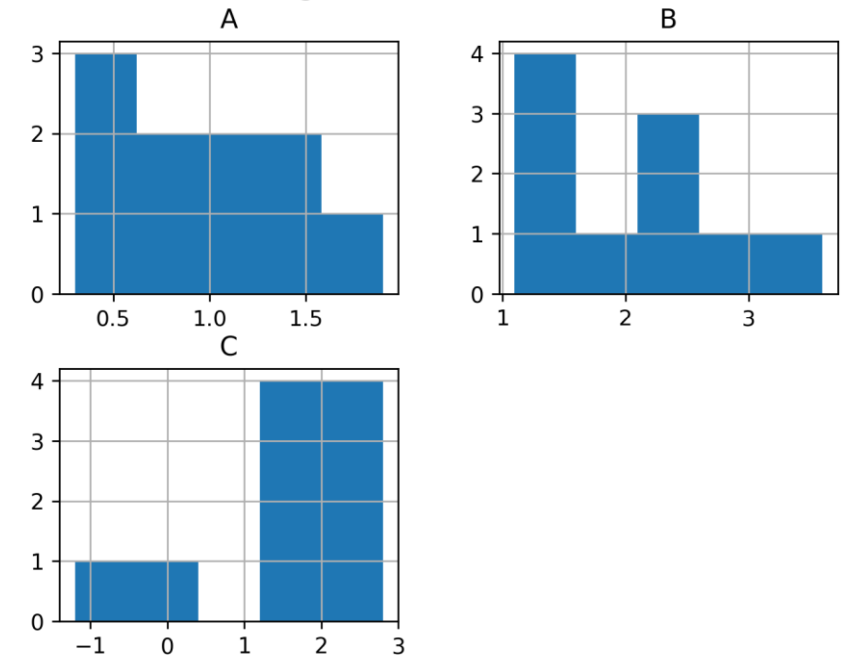
– Seaborn:

E.g. plot for one selected feature

```
# seaborn histogram
plt.figure(figsize=(10, 6)) # width, height in inches for the "canvas"
sns.histplot(data=df["A"], bins=5, kde=True) # kde = kernel density estimate
plt.title('Histogram of Feature A')
plt.show()
```

<https://seaborn.pydata.org/generated/seaborn.histplot.html>

Histograms of All Features



Visualisation – Box Plots

Box plots are a great method to provide an **overview** on the locality, spread and skewness groups of numerical data through their quartiles. Box plots also reveal potential outliers.

Maximum (Q4 / 100th percentile): highest data point (outliers excluded) indicated by “whiskers”

Upper quartile (Q3 / 75th percentile): median of upper half of the dataset, indicated by upper box boundary

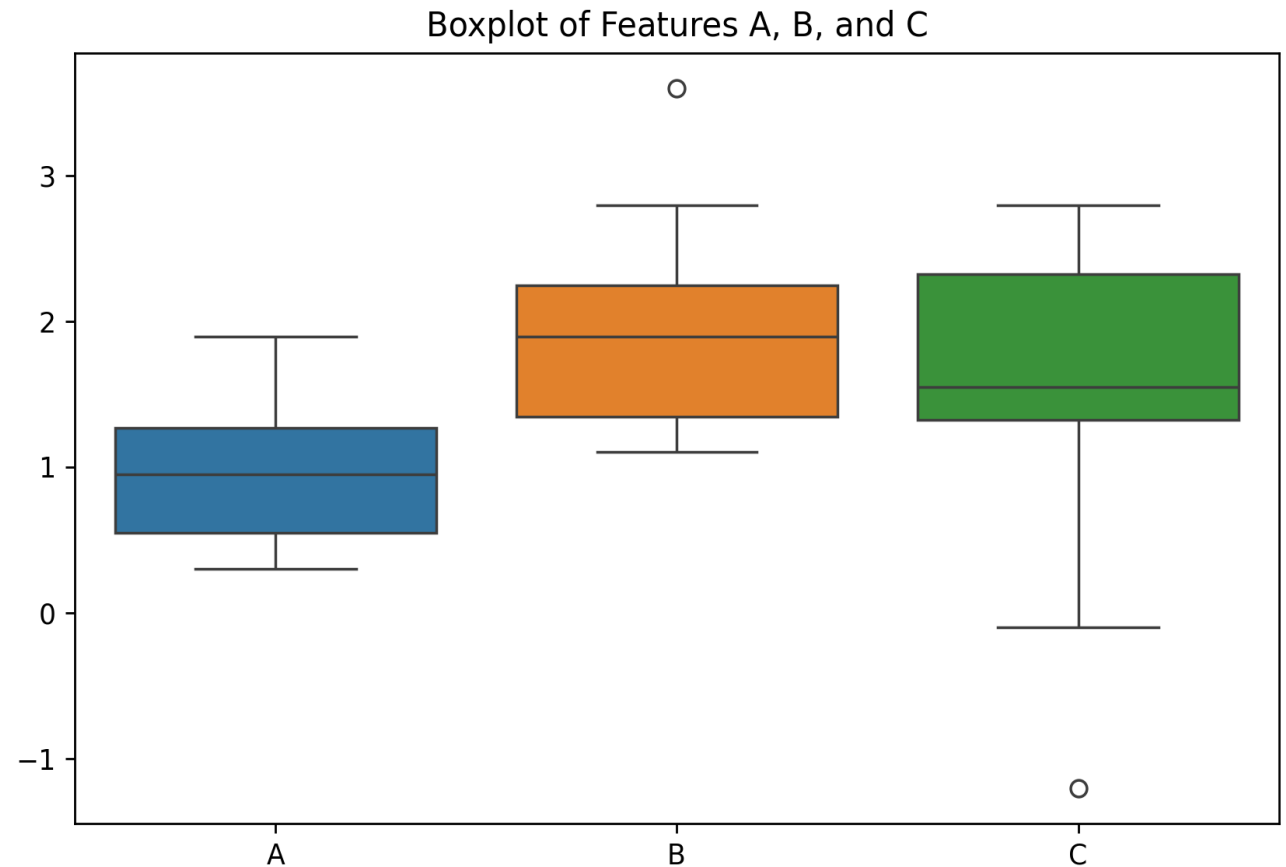
Median (Q2 / 50th percentile): indicated by line in the box

Lower quartile (Q1 / 25th percentile): median of lower half of the dataset, indicated by upper box boundary

Minimum (Q0 / 0th percentile): lowest data point (outliers excluded) indicated by “whiskers”

Interquartile range (IQR) = Q3 – Q1. Usually, $1.5 \times \text{IQR}$ is considered the threshold for **outliers** (marked with small circles)

<https://seaborn.pydata.org/generated/seaborn.boxplot.html>



```
# boxplot
plt.figure(figsize=(8, 5)) # width, height in inches for the "canvas"
sns.boxplot(data=df)
plt.title('Boxplot of Features A, B, and C')
plt.show()
```

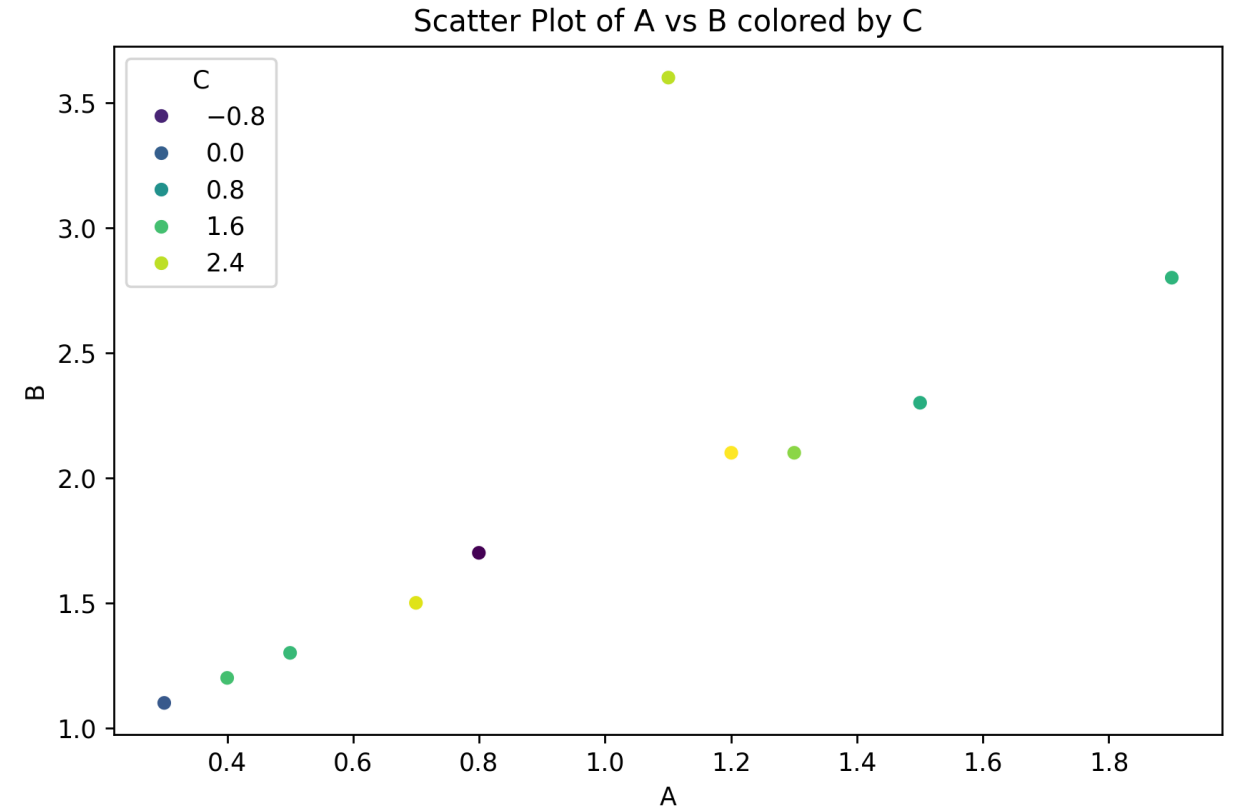
Visualisation – Scatter Plot

Scatter plots are the classic plot to **compare two features** on two axes (**x = “feat1”, y = “feat2”**)

A third dimension can be added by adding a coloured hue (**hue = “feat3”**)

A good choice in colour palette helps visualization if a hue is set.

```
# scatter plot
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='A', y='B', hue='C', palette='viridis')
plt.title('Scatter Plot of A vs B colored by C')
plt.show()
```



<https://seaborn.pydata.org/generated/seaborn.scatterplot.html>

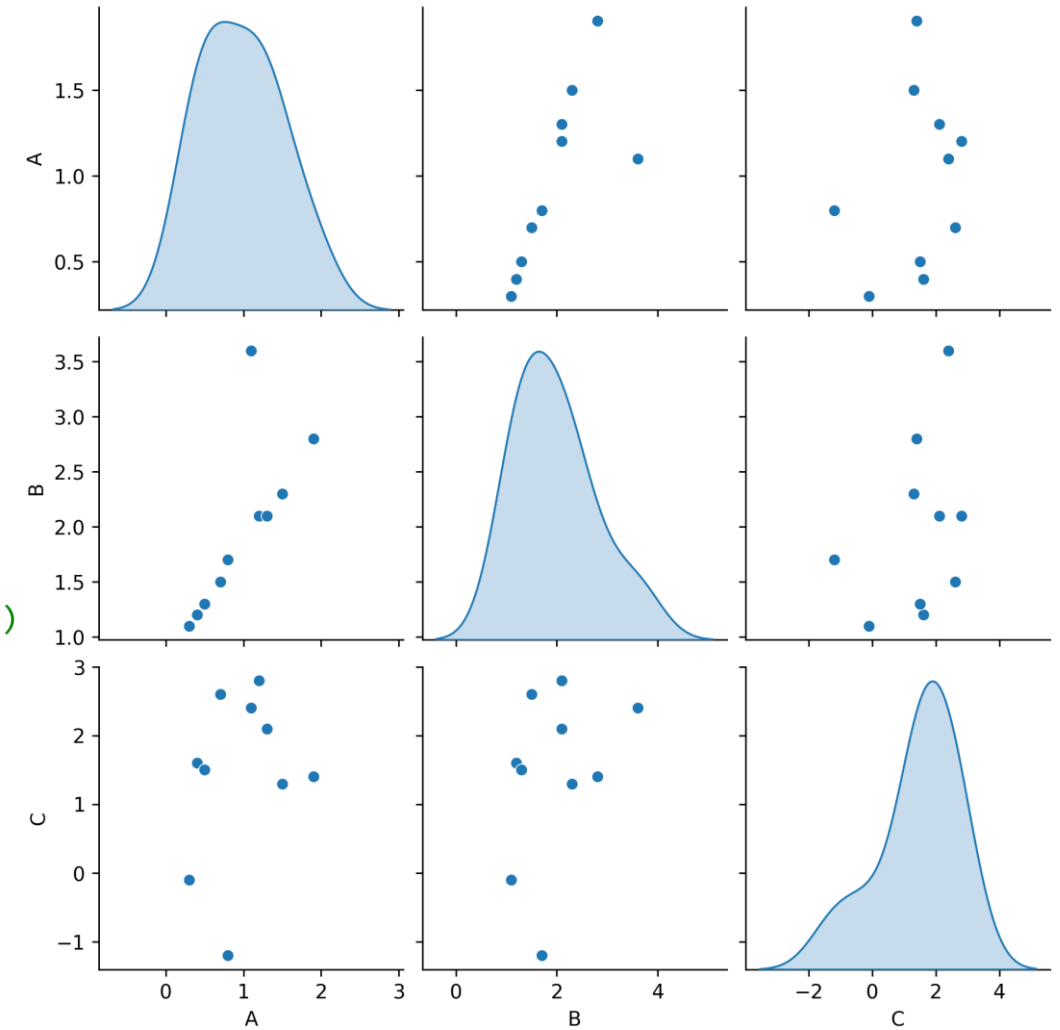
Visualisation – Pair Plot

Pair plot is a method for visualizing **relationships between multiple variables** in a dataset in order to visualize patterns easily

Essentially a grid of scatter plots.

The diagonal can be chosen, e.g. as histograms or kde (kernel density estimate)

```
# pair plot
sns.pairplot(data=df, diag_kind='kde') # diag_kind = type of plot on diagonal'
plt.suptitle('Pairplot of Features A, B, and C', y=1.02)
plt.show()
```



<https://seaborn.pydata.org/generated/seaborn.pairplot.html>

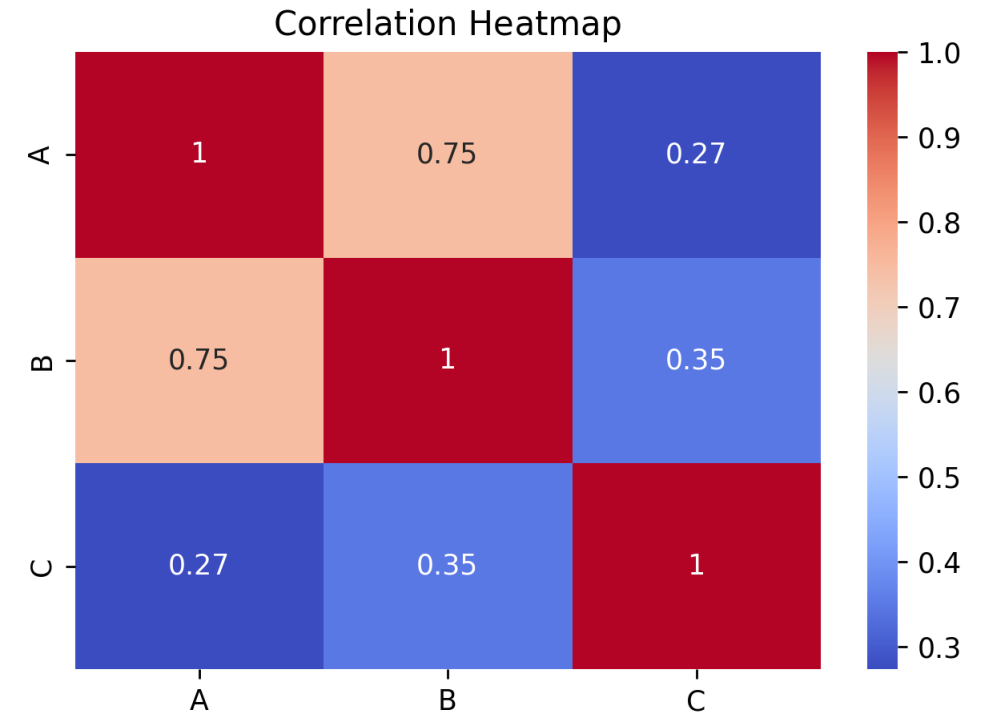
Visualisation – Heatmap

Heat maps plot rectangular data as a colour-encoded matrix where the function for the hue (or intensity) is depending on the purpose.

Very diverse use cases, e.g.

- Correlation between features (here)
- Temperature maps
- Visualising quality of a result (e.g. reaction yield)
- ...

```
# correlation heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

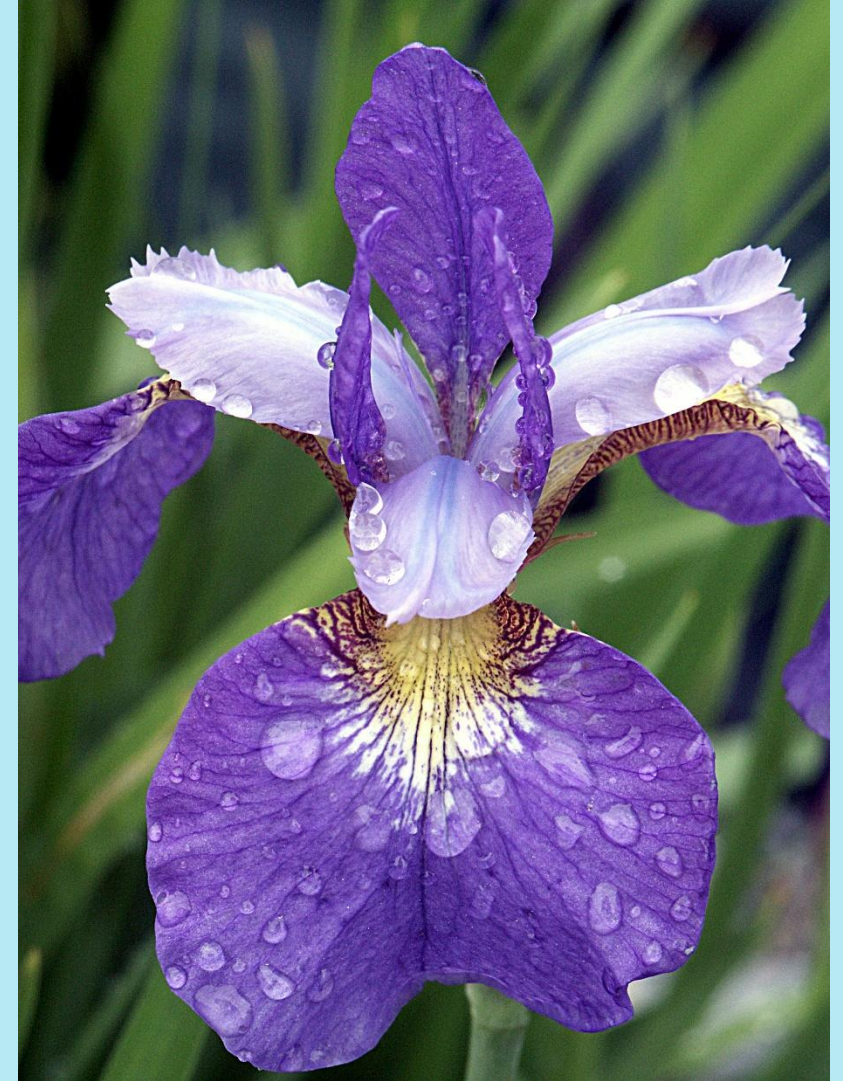


<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Activity: EDA on iris dataset

Have a look at the `eda_iris_part2.ipynb` notebook. Run the following tasks:

- 1) Univariate analysis:
 - Histograms on the components
 - Box plots to identify outliers
- 2) Bivariate analysis:
 - Scatterplots
 - heatmaps to identify correlations
- 3) Grouped analysis and visualisation:
 - Use the `pandas.groupby()` method to assess the mean values for per species
 - Visualise some features grouped by species as box plots
- 4) Which patterns can you identify?



Dimensionality reduction

High dimensional data has some practical drawbacks:

- Difficult visualisation
- Slower training for ML models (more features)
- Overfitting (more noise than patterns)
- Multicollinearity (quite a few features may be correlated)

Dimensionality reduction techniques, reduce number of features while preserving as much information as possible. Basically, two types:

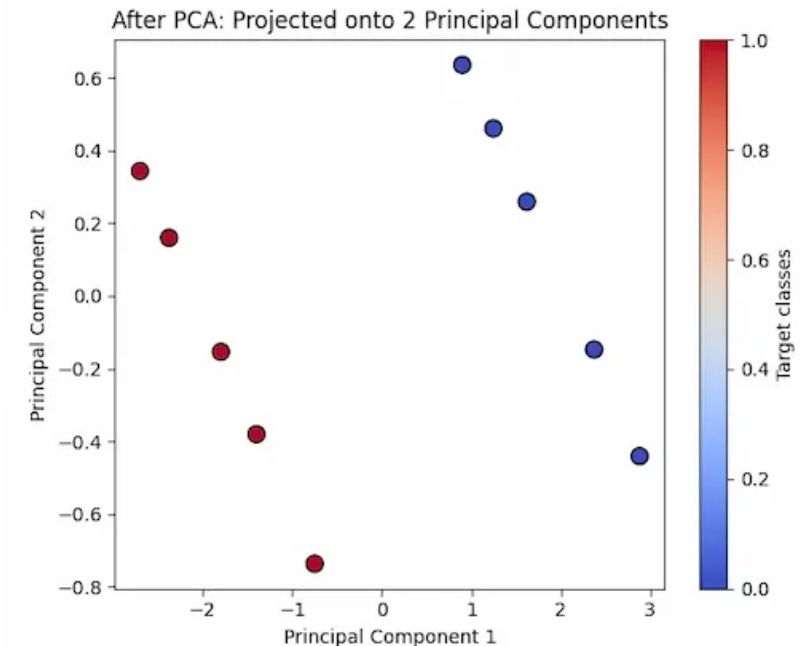
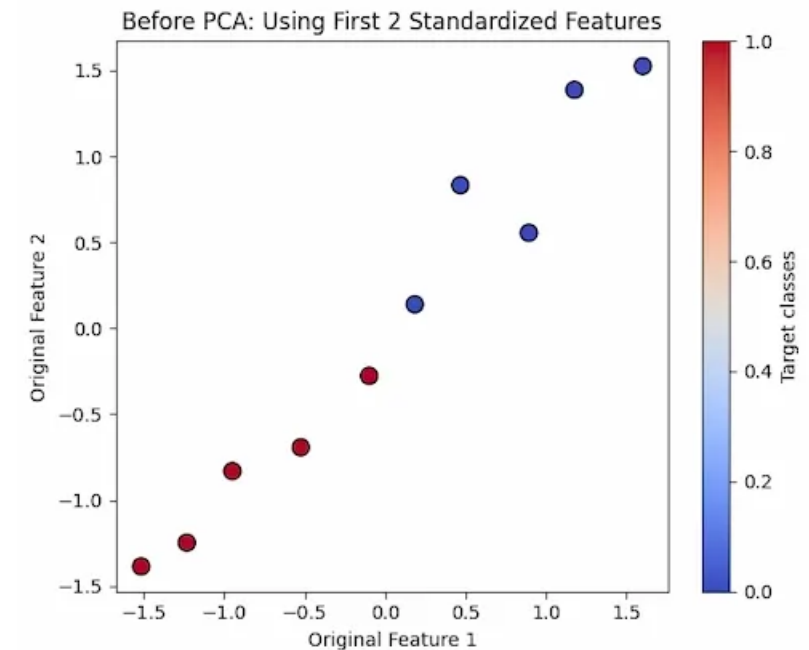
- 1) Feature selection: Original features are preserved, but fewer of them will be taken into account (e.g. Filters)
- 2) Feature extraction: new, compressed representations of the data are obtained by transforming or combining original features to capture most of the variance / information in the data. E.g.:
 - PCA (Principal Component Analysis): most popular method, linear technique
 - t-SNE (t-distributed Stochastic Neighbor Embedding): non-linear technique
 - Autoencoders: neural network-based

Dimensionality reduction: PCA

Working principle:

- 1) Standardizing the data (so each feature has equal importance)
- 2) Calculate covariance matrix to check how features relate to each other. Results in eigenvectors (direction of PC) and eigenvalues (importance)
- 3) Finding directions (“principal components”) where the data varies the most:
 - **1st Principal Component (PC1):** The direction of maximum variance (most spread).
 - **2nd Principal Component (PC2):** The next best direction, *perpendicular to PC1* and so on.
- 4) After finding the components that capture the **maximum variance**, the original data is projected onto these new axes

Graphs from: <https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/>



Activity: EDA on iris dataset

Perform a PCA on the iris data set to visualise all four features in a 2D plot.

Compare to the plots of the bivariate analysis and discuss, which features are showing the highest variance (main features of the primary components).

Discuss how to interpret the PCA plot.

